

Escuela Superior Politécnica del Litoral

Facultad de Ingeniería en Electricidad y Computación

Lenguajes de Programación

Informe Final

PoliLink

Plataforma web para la gestión y difusión centralizada
de eventos de comunidades ESPOL

Integrantes

Darwin Leonardo Díaz González

`dldiaz@espol.edu.ec`

Gabriel Alejandro Tumbaco Santana

`gatumbac@espol.edu.ec`

Profesor

MSc. Rodrigo Saraguro

Guayaquil – Ecuador

19 de agosto de 2026

Índice

1. Problemática a resolver	4
2. Objetivos	4
2.1. Objetivo general	4
2.2. Objetivos específicos	4
3. Alcance del proyecto	4
4. Características de la solución	5
5. Requerimientos funcionales implementados	5
6. Arquitectura de la aplicación	5
6.1. Tipo de aplicación	5
6.2. Patrón de arquitectura	6
6.3. Herramientas y frameworks	6
7. Lenguajes de programación	7
7.1. Back-end: PHP	7
7.2. Front-end: TypeScript	7
8. Diseño de la solución	7
9. Implementación final	8
9.1. Backend	8
9.2. Frontend o Visualización	8
10.Resultados	9
11.Vista general de la solución	10
12.Conclusiones	12
13.Recomendaciones	12
14.Referencias	13

Índice de cuadros

1.	Requerimientos funcionales implementados.	6
2.	Estado final de implementación del backend.	8
3.	Estado final de implementación del frontend.	9

Índice de figuras

1.	Arquitectura de PoliLink.	6
2.	Resultado de Gabriel: gestión de eventos propios del organizador con acceso a edición, cancelación y consulta de inscritos.	9
3.	Resultado de Darwin: consulta de inscripciones, inscritos activos, cupo total y cupos disponibles de un evento.	10
4.	Vista general de la landing: hero público con los accesos principales al catálogo de eventos y al directorio de comunidades.	10
5.	Vista general de la landing: métricas públicas y sección de eventos destacados.	11
6.	Vista general de la landing: comunidades disponibles y explicación de cómo funciona PoliLink.	11
7.	Vista general de la landing: llamado final para explorar eventos, comunidades u organizar una comunidad.	12

1. Problemática a resolver

Los clubes y organizaciones estudiantiles de ESPOL desarrollan actividades académicas, culturales, profesionales y recreativas que aportan a la formación integral de los estudiantes. La institución reconoce estos grupos como espacios de desarrollo multidisciplinario y les brinda apoyo para la realización de actividades [1, 2]. Sin embargo, la difusión de los eventos suele realizarse de forma independiente mediante publicaciones en redes sociales, principalmente Instagram. Como resultado, la información se encuentra dispersa entre perfiles, historias y enlaces externos, por lo que un estudiante puede no enterarse a tiempo de un taller, charla, hackatón o feria de interés.

Esta situación dificulta que la comunidad politécnica conozca oportunamente las actividades disponibles y también limita a los organizadores al momento de concentrar inscripciones y consultar la asistencia esperada. Por ejemplo, cuando un club como TAWS organiza un hackatón, el evento debería poder llegar a estudiantes de distintas carreras que tengan interés en participar. Por esta razón, se propone una plataforma web centralizada que conecte a los clubes y organizaciones con los estudiantes mediante la publicación, consulta e inscripción de eventos.

2. Objetivos

2.1. Objetivo general

Desarrollar una aplicación web que centralice la publicación, consulta e inscripción de eventos organizados por las comunidades estudiantiles de ESPOL.

2.2. Objetivos específicos

1. Implementar un módulo para que los organizadores publiquen directamente, modifiquen y cancelen eventos con información de fecha, ubicación, categoría y cupos disponibles.
2. Desarrollar un catálogo que permita a los estudiantes consultar y filtrar eventos por fecha, categoría, comunidad organizadora y modalidad.
3. Implementar el proceso de inscripción y cancelación de inscripción, junto con la consulta de inscritos para cada organizador.

3. Alcance del proyecto

El proyecto consiste en un prototipo funcional de aplicación web dirigido a los estudiantes, organizadores y administradores de comunidades estudiantiles de ESPOL. La solución trabaja con cuentas locales y permite registrar comunidades, publicar eventos, consultar actividades, gestionar inscripciones y solicitar la incorporación a comunidades.

La información de cada evento incluye título, descripción, categoría, fecha, hora, ubicación, modalidad, cupo y comunidad responsable.

Los organizadores pueden publicar, editar y cancelar sus propios eventos directamente, sin aprobación administrativa de cada evento. Para la creación de comunidades existe un flujo separado de solicitud y revisión administrativa, mientras que la incorporación a una comunidad puede requerir aprobación del organizador responsable. No se integrará con los sistemas institucionales de autenticación, correo, calendario ni gestión académica; tampoco se procesarán pagos ni se validará la asistencia mediante códigos QR. Estas funcionalidades pueden considerarse como trabajo futuro. El mapa público del Campus Gustavo Galindo se utiliza únicamente como referencia para mostrar la ubicación textual de los eventos [3].

4. Características de la solución

La versión entregable cuenta con las siguientes características principales:

- Catálogo centralizado de eventos publicados por clubes y organizaciones estudiantiles de ESPOL.
- Búsqueda y filtros por fecha, categoría, modalidad y comunidad organizadora.
- Vista de detalle con descripción, ubicación, cupos y enlace de inscripción.
- Publicación directa, edición y cancelación de eventos por parte del organizador.
- Inscripción y cancelación de inscripción por parte del estudiante.
- Consulta de la lista de inscritos y de los cupos disponibles para cada evento.
- Landing pública y directorio de comunidades con acceso a sus perfiles.
- Solicitud y revisión de membresías de comunidades.
- Panel administrativo para aprobar comunidades y mantener categorías, modalidades y ubicaciones.
- Interfaz responsive con estados de carga, vacío, error y confirmación.

5. Requerimientos funcionales implementados

Los requerimientos se distribuyen de modo que cada integrante implemente funcionalidades de lectura y escritura tanto en el frontend como en el backend. El inicio de sesión, el diseño visual y la gestión de la base de datos se consideran componentes de apoyo y no constituyen requerimientos asignados.

6. Arquitectura de la aplicación

6.1. Tipo de aplicación

PoliLink es una aplicación web responsive. Este tipo de aplicación permite que estudiantes y organizadores accedan desde un navegador en computadora o teléfono sin

Requerimiento	Descripción	Responsable
Crear, editar y cancelar eventos	Permite al organizador publicar directamente un evento, modificar su información o cancelar una publicación.	Gabriel Tumbaco
Consultar y filtrar eventos	Permite al estudiante visualizar el catálogo y aplicar filtros por fecha, categoría, modalidad y comunidad organizadora.	Gabriel Tumbaco
Inscribirse y cancelar inscripción	Permite al estudiante reservar un cupo disponible o retirar su inscripción de un evento.	Darwin Díaz
Consultar inscritos y cupos disponibles	Permite al organizador revisar los estudiantes registrados y la disponibilidad de su evento.	Darwin Díaz
Descubrir comunidades y solicitar membresía	Permite al estudiante consultar comunidades públicas y solicitar su incorporación a una comunidad.	Darwin Díaz
Revisar solicitudes y administrar catálogos	Permite al administrador aprobar comunidades y mantener categorías, modalidades y ubicaciones.	Darwin Díaz
Presentación pública y navegación de la plataforma	Permite descubrir eventos y comunidades desde una landing responsive con enlaces al catálogo.	Gabriel Tumbaco

Cuadro 1: Requerimientos funcionales implementados.

requerir la instalación de una aplicación móvil. Además, facilita la publicación rápida de información y la consulta de eventos antes de que se realicen.

6.2. Patrón de arquitectura

La aplicación utiliza una arquitectura cliente-servidor con el patrón Modelo–Vista–Controlador (MVC). El frontend implementado en TypeScript consume una API REST desarrollada en PHP. El backend concentra las reglas de negocio y el acceso a la base de datos, mientras que la interfaz presenta la información y envía las solicitudes de los usuarios.

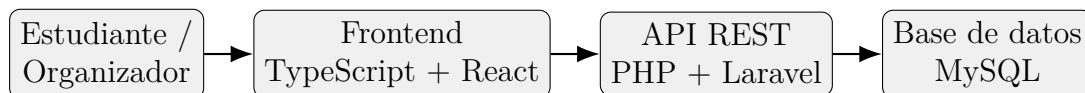


Figura 1: Arquitectura de PoliLink.

6.3. Herramientas y frameworks

- **Framework frontend:** React 19.2.8 con TypeScript 6.0.2 para construir una interfaz reutilizable y con validación estática de tipos [6, 5].

- **Herramientas frontend:** Vite 8.2.0, React Router 8.3.0, TanStack Query 5.101.4 y Vitest 4.1.10 para desarrollo, navegación, consultas y pruebas automatizadas [7].
- **Framework backend:** Laravel 13.24.0 sobre PHP 8.3, con Laravel Sanctum 4.3.3 para sesiones locales y protección de solicitudes [8, 9].
- **API:** API REST propia para administrar eventos, comunidades, inscripciones, membresías, solicitudes administrativas y catálogos.
- **Persistencia y colaboración:** MySQL para persistencia de datos y Git/GitHub para control de versiones y trabajo colaborativo [10, 11].

7. Lenguajes de programación

7.1. Back-end: PHP

PHP se utiliza en el backend mediante Laravel. El lenguaje se adapta al desarrollo de aplicaciones web dinámicas, cuenta con una sintaxis accesible y dispone de un ecosistema amplio para trabajar con rutas, controladores, validaciones y bases de datos [4]. Su tipado dinámico favorece el desarrollo rápido de prototipos y su integración con HTML continúa siendo una característica útil en aplicaciones web.

Como desventaja, el tipado dinámico puede permitir errores de tipos durante la ejecución si no se aplican validaciones y pruebas adecuadas. Además, PHP admite diferentes estilos de programación, por lo que es necesario mantener convenciones claras para conservar la legibilidad del código.

7.2. Front-end: TypeScript

TypeScript se utiliza en el frontend junto con React. TypeScript amplía JavaScript mediante tipos estáticos, lo cual permite detectar inconsistencias antes de ejecutar el programa y facilita el mantenimiento de componentes, formularios y respuestas de la API [5]. Su compatibilidad con JavaScript permite utilizar las bibliotecas del ecosistema web sin abandonar las ventajas de la verificación de tipos.

Como desventaja, TypeScript requiere una etapa de compilación a JavaScript y demanda definir interfaces y tipos adicionales. Esto puede aumentar el tiempo inicial de desarrollo, aunque reduce errores cuando el proyecto crece.

8. Diseño de la solución

Herramienta de diseño utilizada: Figma [12].

Durante la etapa de diseño se elaboraron bocetos de baja fidelidad para el catálogo de eventos, el detalle de evento, el panel del organizador y el formulario de creación. Estos bocetos sirvieron como referencia inicial para definir la navegación y la jerarquía de información. En el informe final se conservan las decisiones de diseño y se presentan

únicamente resultados de la aplicación implementada, de acuerdo con los requisitos de la entrega.

9. Implementación final

9.1. Backend

El backend final de PoliLink se desarrolló con Laravel, MySQL y una API REST. La aplicación cuenta con autenticación local mediante Laravel Sanctum, catálogo público de eventos, directorio de comunidades, gestión de eventos e imágenes, inscripciones, membresías, solicitudes de creación de comunidades y administración de catálogos. El repositorio del grupo se encuentra en <https://github.com/Gatumbac/PoliLink>. Los enlaces directos a las carpetas entregables son <https://github.com/Gatumbac/PoliLink/tree/main/backend> y <https://github.com/Gatumbac/PoliLink/tree/main/frontend>.

Implementación	Responsable	Estado
Catálogo público de eventos, búsqueda, filtros y datos de referencia para formularios.	Gabriel Tumbaco	Hecho
Autenticación local con Laravel Sanctum, registro, inicio y cierre de sesión.	Gabriel Tumbaco	Hecho
Gestión de comunidades y consulta de comunidades y eventos propios.	Gabriel Tumbaco	Hecho
Creación, edición y cancelación de eventos propios mediante sesión autenticada.	Gabriel Tumbaco	Hecho
Carga, reemplazo y eliminación de imágenes de portada de eventos.	Gabriel Tumbaco	Hecho
Inscripción o reactivación de inscripción en eventos.	Darwin Díaz	Hecho
Cancelación de inscripción y liberación de cupos.	Darwin Díaz	Hecho
Consulta de inscritos, cupos disponibles y mis inscripciones.	Darwin Díaz	Hecho
Directorio público y detalle de comunidades.	Darwin Díaz	Hecho
Solicitud y aprobación o rechazo de membresías.	Darwin Díaz	Hecho
Solicitudes administrativas de creación de comunidades y catálogo de referencia.	Darwin Díaz	Hecho

Cuadro 2: Estado final de implementación del backend.

El repositorio incluye pruebas automatizadas y colecciones de solicitudes para validar los contratos principales de la API. Las capturas del informe final se concentran en la sección de resultados para cumplir el límite de una evidencia frontend por integrante.

9.2. Frontend o Visualización

El frontend final de PoliLink se desarrolló con React, TypeScript y Vite. La interfaz utiliza componentes reutilizables, validación de formularios, consultas cacheadas y un diseño responsive para sus diferentes vistas. La implementación final incluye el catálogo y detalle de eventos, gestión de eventos del organizador, inscripciones, directorio de comunidades, membresías, panel administrativo y una landing pública interactiva.

Implementación	Responsable	Estado
Configuración del frontend, componentes compartidos y estructura de la aplicación.	Gabriel Tumbaco	Hecho
Catálogo público, búsqueda, filtros y detalle de eventos.	Gabriel Tumbaco	Hecho
Gestión de eventos del organizador: comunidades, creación, edición, imágenes y cancelación.	Gabriel Tumbaco	Hecho
Landing pública interactiva con métricas, eventos y comunidades reales.	Gabriel Tumbaco	Hecho
Creación del scaffold inicial del monorepo y la aplicación frontend.	Darwin Díaz	Hecho
Integración de inscripciones, mis inscripciones, asistentes y cupos.	Darwin Díaz	Hecho
Directorio público y detalle de comunidades.	Darwin Díaz	Hecho
Solicitud y revisión de membresías de comunidades.	Darwin Díaz	Hecho
Panel administrativo para solicitudes de comunidades y catálogos.	Darwin Díaz	Hecho

Cuadro 3: Estado final de implementación del frontend.

10. Resultados

Para el informe final se presenta una evidencia frontend por integrante y cuatro capturas generales de la landing. Cada figura debe mostrar una funcionalidad final ejecutándose en el navegador, con datos reales o de demostración y sin herramientas de desarrollo visibles.

Figura 2: Resultado de Gabriel: gestión de eventos propios del organizador con acceso a edición, cancelación y consulta de inscritos.

La evidencia de la figura 2 muestra una actividad publicada por el organizador y las acciones disponibles para editarla, cancelarla o consultar sus inscritos.

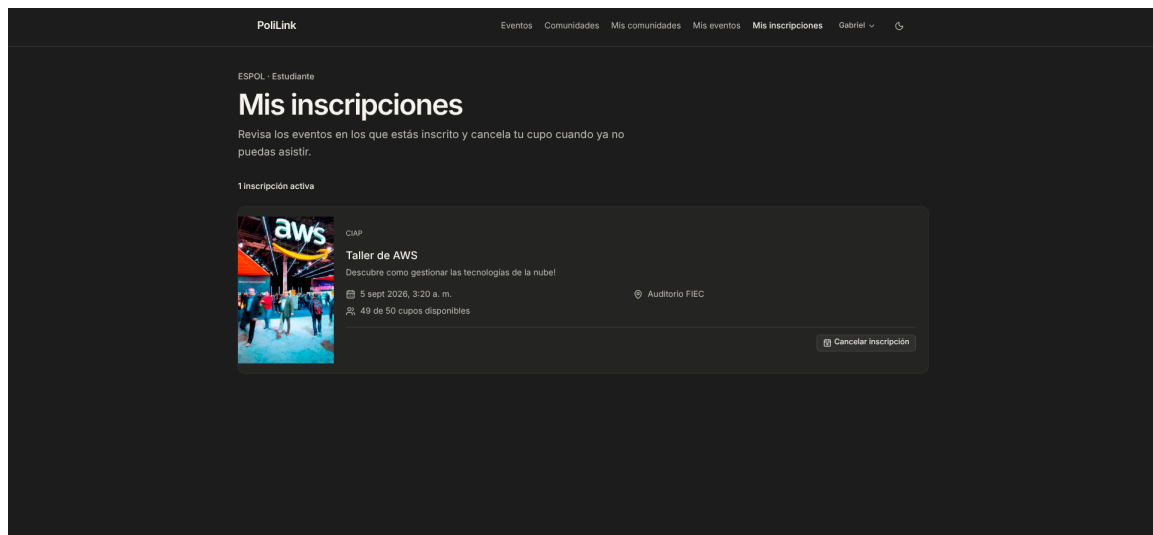


Figura 3: Resultado de Darwin: consulta de inscripciones, inscritos activos, cupo total y cupos disponibles de un evento.

La evidencia de la figura 3 muestra la funcionalidad de inscripciones desarrollada por Darwin desde la vista del organizador. La pantalla permite consultar los inscritos activos, el cupo total y los cupos disponibles del evento.

11. Vista general de la solución

Además de las evidencias individuales, se incluyen cuatro capturas generales de la landing para mostrar sus secciones principales. Estas imágenes son complementarias y no sustituyen la evidencia de cada integrante.

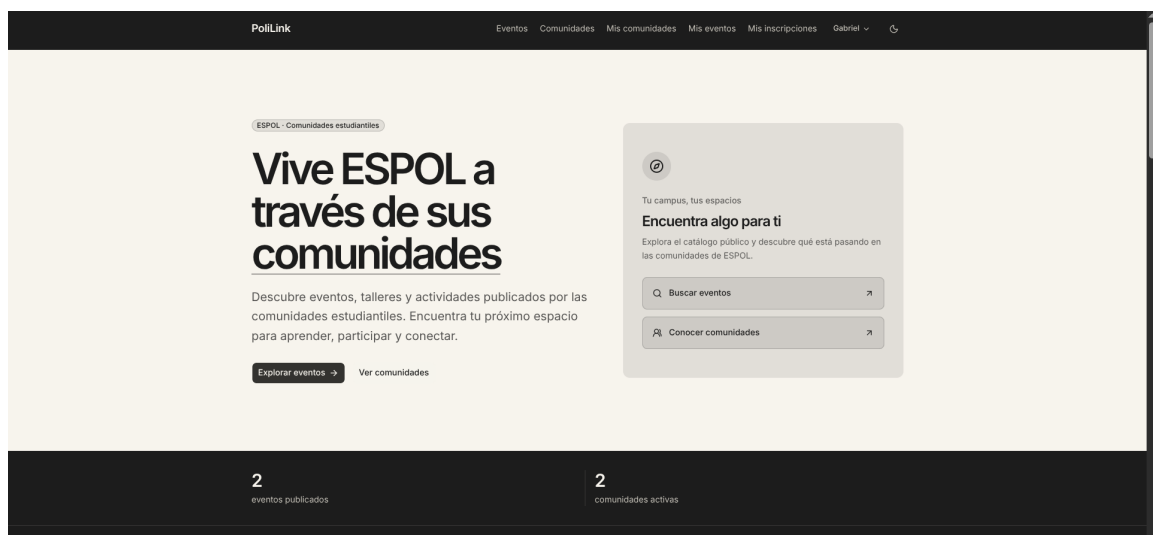


Figura 4: Vista general de la landing: hero público con los accesos principales al catálogo de eventos y al directorio de comunidades.

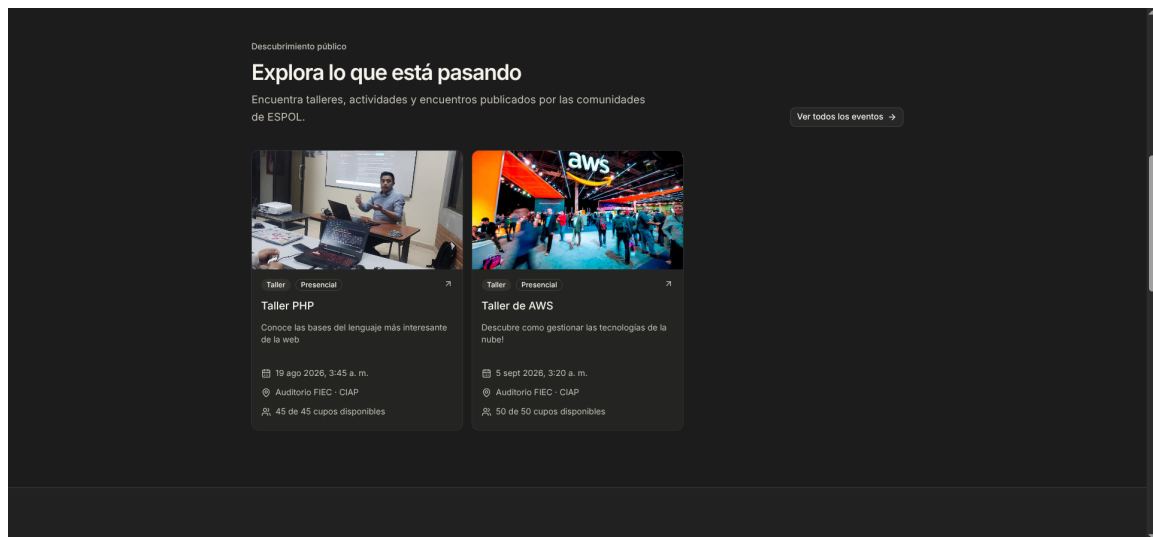


Figura 5: Vista general de la landing: métricas públicas y sección de eventos destacados.

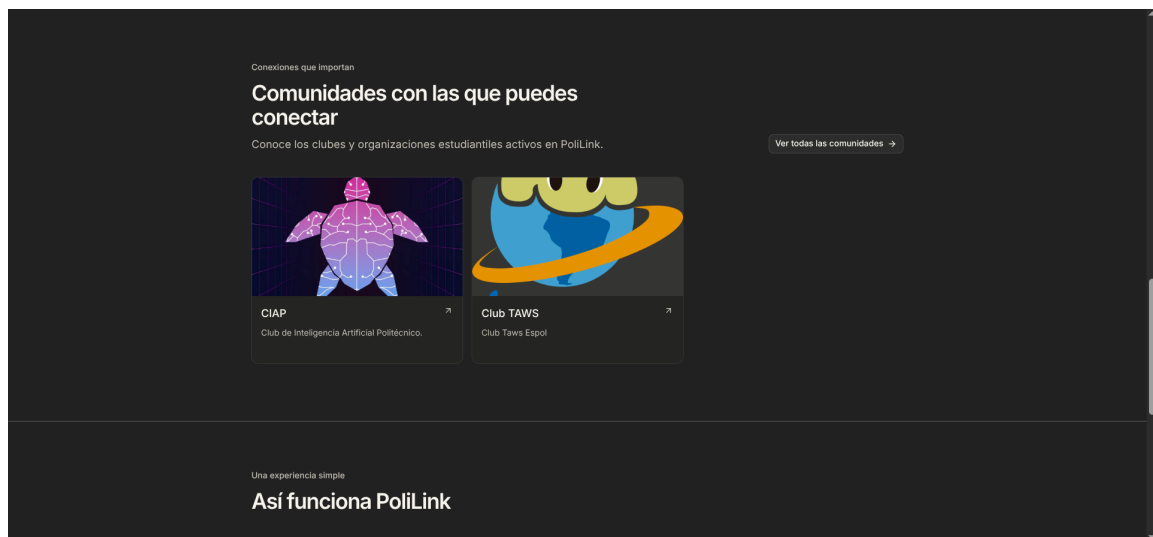


Figura 6: Vista general de la landing: comunidades disponibles y explicación de cómo funciona PoliLink.

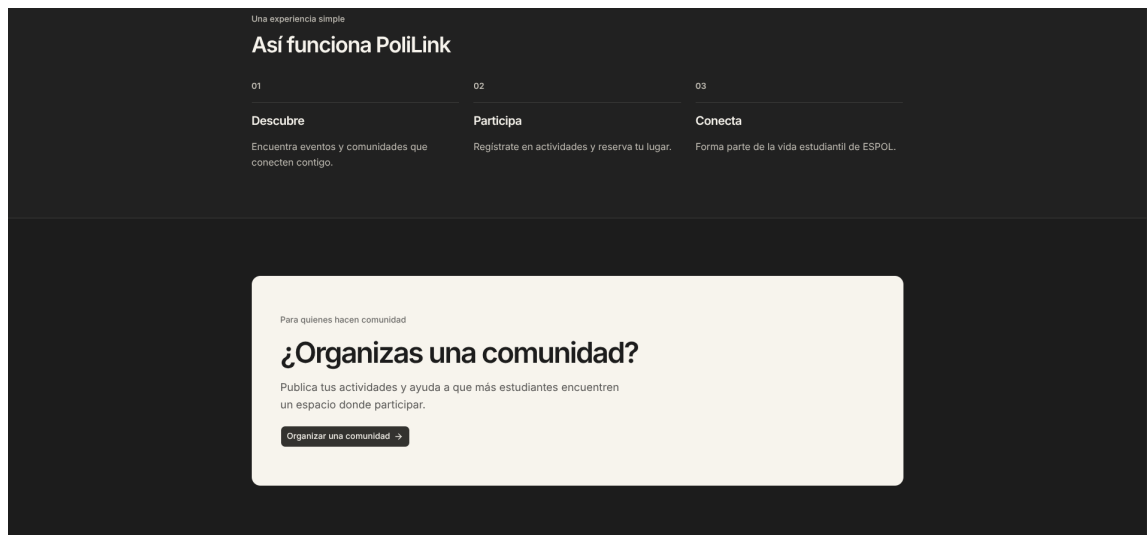


Figura 7: Vista general de la landing: llamado final para explorar eventos, comunidades u organizar una comunidad.

Las cuatro capturas generales muestran la landing completa por secciones, sin repetir la evidencia individual de Gabriel.

12. Conclusiones

Gabriel Tumbaco. El desarrollo de PoliLink permitió aplicar React, TypeScript, Vite y Laravel en una solución integrada para publicar y descubrir eventos estudiantiles. La construcción del catálogo, la gestión de eventos y la landing mostró la importancia de mantener contratos claros entre el frontend y la API, reutilizar componentes y diseñar estados de carga, error y vacío desde el inicio.

Darwin Díaz. La implementación de inscripciones, asistentes, membresías y administración permitió comprender cómo las reglas de autorización y los estados del backend deben reflejarse en la interfaz. La integración de React con Laravel y MySQL también evidenció la necesidad de validar permisos, cupos y transiciones de estado para evitar operaciones inconsistentes.

13. Recomendaciones

Gabriel Tumbaco. Como mejora futura se recomienda integrar la autenticación institucional de ESPOL y desplegar frontend, backend y base de datos en un entorno controlado con variables de configuración separadas para desarrollo y producción.

Darwin Díaz. Se recomienda incorporar notificaciones por correo o calendario para confirmar inscripciones, avisar cambios de eventos y comunicar decisiones sobre membresías. También sería conveniente ampliar las pruebas de extremo a extremo con servicios reales y agregar monitoreo para detectar errores de integración.

14. Referencias

Referencias

- [1] Escuela Superior Politécnica del Litoral, “Clubes y agrupaciones estudiantiles.” [En línea]. Disponible en: <https://www.espol.edu.ec/es/clubes-y-agrupaciones-estudiantiles>. [Accedido: 19-ago-2026].
- [2] Unidad de Bienestar Politécnico, “Programa de Apoyo de Clubes Estudiantiles y Organizaciones Estudiantiles Profesionales.” [En línea]. Disponible en: <https://oe.espol.edu.ec/programa-de-apoyo/>. [Accedido: 19-ago-2026].
- [3] Escuela Superior Politécnica del Litoral, “Mapa del campus.” [En línea]. Disponible en: <https://www.espol.edu.ec/es/mapa-del-campus>. [Accedido: 19-ago-2026].
- [4] PHP Documentation Group, “PHP: Hypertext Preprocessor.” [En línea]. Disponible en: <https://www.php.net/manual/es/intro-what-is.php>. [Accedido: 19-ago-2026].
- [5] Microsoft, “TypeScript: JavaScript with syntax for types.” [En línea]. Disponible en: <https://www.typescriptlang.org/>. [Accedido: 19-ago-2026].
- [6] Meta Open Source, “React: The library for web and native user interfaces.” [En línea]. Disponible en: <https://react.dev/>. [Accedido: 19-ago-2026].
- [7] Vite Team, “Vite: Next generation frontend tooling.” [En línea]. Disponible en: <https://vite.dev/>. [Accedido: 19-ago-2026].
- [8] Laravel, “Laravel 13.x documentation.” [En línea]. Disponible en: <https://laravel.com/docs/13.x>. [Accedido: 19-ago-2026].
- [9] Laravel, “Laravel Sanctum documentation.” [En línea]. Disponible en: <https://laravel.com/docs/13.x/sanctum>. [Accedido: 19-ago-2026].
- [10] Oracle Corporation, “MySQL 8.4 Reference Manual.” [En línea]. Disponible en: <https://dev.mysql.com/doc/>. [Accedido: 19-ago-2026].
- [11] Software Freedom Conservancy, “Git documentation.” [En línea]. Disponible en: <https://git-scm.com/doc>. [Accedido: 19-ago-2026].
- [12] Figma, “Figma: The collaborative interface design tool.” [En línea]. Disponible en: <https://www.figma.com/>. [Accedido: 19-ago-2026].